

Composants Web personnalisés

Les composants Web personnalisés (Web Components) permettent de créer vos propres balises HTML réutilisables avec leurs propres fonctionnalités et styles.

Pourquoi les utiliser ?

- **Réutilisabilité** : Créez un composant une fois, utilisez-le partout
- **Encapsulation** : Les styles du composant n'affectent pas le reste de la page
- **Standard Web** : Pas besoin de framework, ça fonctionne nativement dans le navigateur

Les 2 technologies principales

1. **Custom Elements** : Créer des balises HTML personnalisées
2. **Shadow DOM** : Isoler les styles et le contenu

Voyez le Custom Element comme la coquille d'un composant, tandis que le Shadow DOM est le contenu à l'intérieur de cette coquille.

1. Custom Elements (Éléments personnalisés)

Créer une balise HTML personnalisée en héritant de `HTMLElement`. Cela permet d'avoir, par exemple, accès aux écouteurs d'événements et au cycle de vie du composant. Vous devez appeler le constructeur de la classe parente avec `super()`. Vous pouvez ensuite importer tout simplement le fichier JS contenant le composant pour l'utiliser dans votre page.

Règle importante

Le nom doit **obligatoirement contenir un tiret (-)** pour éviter les conflits avec les balises HTML natives. Ce nom est défini lors de l'enregistrement du composant avec `customElements.define()`.

Exemples : `<user-card>`, `<my-button>`, `<loading-spinner>`

Exemple simple

```
class UserCard extends HTMLElement {
  constructor() {
    super();
    this.afficher();
  }

  afficher() {
    this.innerHTML = `
      <div class="user-card">
        <h3>Nom d'utilisateur</h3>
        <p>Description</p>
      </div>
    `;
  }
}
```

```
// Enregistrer le composant
customElements.define("user-card", UserCard);
export default UserCard;
```

Utilisation :

```
// Dans le fichier js principal
import "../chemin/vers/UserCard.js";
```

```
<user-card></user-card>
```

Méthodes de cycle de vie (les plus utiles)

Le cycle de vie d'un composant web personnalisé comprend plusieurs étapes, chacune pouvant être gérée via des méthodes spécifiques.

Les méthodes suivantes sont toujours disponibles dans une classe qui étend `HTMLElement`. Elles permettent de gérer le comportement du composant à différents moments de son existence.

- `connectedCallback()` : Appelée quand l'élément est ajouté au DOM
- `disconnectedCallback()` : Appelée quand l'élément est retiré du DOM
- `attributeChangedCallback(name, oldValue, newValue)` : Appelée quand un attribut observé change
- `get observedAttributes()` : Définit quels attributs doivent être surveillés pour les changements

```
class MyComponent extends HTMLElement {
  constructor() {
    super();
    // Initialisation de base
    this.#afficher();
  }

  // Appelé quand l'élément est ajouté au DOM
  connectedCallback() {
    // Idéal pour ajouter du contenu ou des événements
    this.addEventListener("click", this.#onClick.bind(this));
  }

  // Appelé quand l'élément est retiré du DOM
  disconnectedCallback() {
    // Nettoyer les événements, timers, etc.
    this.removeEventListener("click", this.#onClick.bind(this));
  }

  #onClick() {
    alert("Composant cliqué!");
  }
}
```

```

    }

    #afficher() {
        this.innerHTML = `

Composant initialisé</p>`;
    }
}


```

Réagir aux attributs

Un des points forts des Custom Elements est la possibilité d'attacher de l'information via des attributs HTML et de réagir aux changements de ces attributs.

Les attributs HTML peuvent être surveillés pour déclencher des actions lorsque leur valeur change. Pour cela, on utilise la méthode statique `observedAttributes` et la méthode `attributeChangedCallback`.

Par exemple, un composant qui affiche un message de bienvenue personnalisé en fonction d'un attribut `name` : Lorsqu'on modifie l'attribut `name`, le message se met à jour automatiquement.

On utilise deux méthodes spéciales : `observedAttributes` pour définir quels attributs surveiller, et `attributeChangedCallback` pour réagir aux changements.

On passe ensuite l'attribut `name` pour directement sur le composant dans la page HTML.

```

class GreetingCard extends HTMLElement {
    constructor() {
        super();
        this.#afficher();
    }

    static get observedAttributes() {
        return ["name"]; // Attributs à surveiller
    }

    attributeChangedCallback(name, oldValue, newValue) {
        // Appelé quand un attribut observé change
        this.#afficher();
    }

    #afficher() {
        const name = this.getAttribute("name") || "Invité";
        this.innerHTML = `

Bonjour, ${name}!</p>`;
    }
}

customElements.define("greeting-card", GreetingCard);


```

Utilisation :

```
<greeting-card name="Marie"></greeting-card>
```

```
<script>
  const card = document.querySelector("greeting-card");
  card.name = "Jean"; // Met à jour le message automatiquement
</script>
```

2. Shadow DOM

Le Shadow DOM permet d'isoler les styles et le contenu d'un composant. Les styles à l'intérieur n'affectent pas le reste de la page, et vice versa. Le Shadow DOM est créé en appelant `attachShadow({ mode: "open" })` dans le constructeur du composant. Cela crée une racine de Shadow DOM où vous pouvez ajouter du HTML et des styles encapsulés. Quand le mode est "open", on peut accéder au Shadow DOM via la propriété `shadowRoot` depuis l'instance du composant.

Pourquoi c'est utile ?

- Pas de conflits de classes CSS
- Styles encapsulés dans le composant
- Code plus propre et organisé

Shadow root

Le Shadow root est la racine du Shadow DOM pour un élément. C'est à partir de cette racine que tous les styles et le contenu encapsulés sont définis. On peut y ajouter des balises `<style>` pour définir des styles spécifiques au composant, ainsi que du HTML pour la structure du composant.

Exemple de base

```
class FancyButton extends HTMLElement {
  constructor() {
    super();

    // Créer un Shadow DOM
    this.attachShadow({ mode: "open" });
    this.afficher();
  }
  afficher() {
    this.shadowRoot.innerHTML = `
      <style>
        button {
          background: #007bff;
          color: white;
          padding: 10px 20px;
          border: none;
          border-radius: 5px;
          cursor: pointer;
        }
        button:hover {
          background: #0056b3;
        }
      </style>
      <button>
```

```

        <slot>Cliquez-moi</slot>
      </button>
    `;
  }
}

customElements.define("fancy-button", FancyButton);
export default FancyButton;

```

Utilisation des slots

Les **slots** permettent d'insérer du contenu HTML plus complexe depuis l'extérieur. On utilise la balise `<slot>` dans le Shadow DOM pour définir où le contenu externe sera inséré et dans l'implémentation du composant dans le HTML, on remplace par le contenu de notre choix.

```

<fancy-button>
  <span style="font-weight: bold; color: yellow;">Mon bouton personnalisé</span>
</fancy-button>
<fancy-button>Envoyer</fancy-button>

```

Slots nommés

Vous pouvez ajouter un attribut `name` aux slots pour définir des emplacements spécifiques pour différents types de contenu.

```

class UserProfile extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: "open" });

    this.shadowRoot.innerHTML = `
      <div class="profile">
        <div class="avatar">
          <slot name="avatar">👤</slot>
        </div>
        <div class="name">
          <slot name="name">Nom</slot>
        </div>
        <div class="bio">
          <slot name="bio">Biographie</slot>
        </div>
      </div>
    `;
  }
}

customElements.define("user-profile", UserProfile);

```

Utilisation :

```
<user-profile>
  <span slot="avatar">👤</span>
  <span slot="name">Marie Dupont</span>
  <span slot="bio">Développeuse Web passionnée</span>
</user-profile>
```

Modifier le CSS d'un composant personnalisé

Par défaut, les styles définis à l'intérieur du Shadow DOM d'un composant personnalisé sont encapsulés et n'affectent pas le reste de la page. Cependant, il est possible de modifier les styles d'un composant personnalisé en utilisant des variables CSS (custom properties).

Vous pouvez utiliser la pseudo-classe `:host` pour appliquer des styles globaux au composant.

```
:host {
  --main-color: blue;
  color: var(--main-color);
  background: var(--main-bg, white);
}
```

Importer un fichier CSS dans le Shadow DOM

Vous pouvez importer un fichier CSS externe dans le Shadow DOM en utilisant la balise `link` à l'intérieur du Shadow DOM. Cela permet de garder vos styles organisés et réutilisables.

```
class LoadingSpinner extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: "open" });

    // Créer la structure du spinner
    this.shadowRoot.innerHTML = `
      <link rel="stylesheet"
href="./assets/js/components/Spinner/spinner.css">

      <div class="spinner-overlay">
        <div class="spinner-container">
          <div class="spinner"></div>
          <div class="message">
            <slot>Chargement en cours...</slot>
          </div>
        </div>
      </div>
    `;
  }
}
```

```
//Restes des méthodes du composant  
}
```

```
:host {  
  --spinner-size: 50px;  
  --spinner-color: #3498db;  
  --spinner-overlay-bg: rgba(0, 0, 0, 0.5);  
}  
  
.spinner-overlay {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  background: var(--spinner-overlay-bg, rgba(0, 0, 0, 0.5));  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  z-index: 1000;  
}  
/*... autres styles pour le spinner ...*/
```